

내부 원장 필요성과 데이터 모델 설계

온체인만으로 부족한 이유

블록체인 4가지 한계 · 오프체인 원장의 역할 · 4개 테이블 설계 · LedgerService 전체 구현
MintStatus 7종 · VALID_TRANSITIONS · ON CONFLICT DO NOTHING · 데이터 수명 관리

LedgerService

Phase 1 VASP 위탁 아키텍처

블록체인만 쓰면 안 되는 4가지 이유

"NFT 소유권은 블록체인에 다 있으니 DB가 필요 없지 않나?" — 이 생각이 왜 틀렸는가

① 조회 성능 — 100배 차이

`balanceOf(userAddress)` 호출 1번 = RPC 네트워크 경유
응답시간 200~500ms → 앱 NFT 목록 로딩 2~3초
DB 조회 = 5ms 이하. 초당 수천 요청도 처리 가능

③ "진행 중" 상태 추적 불가

TX를 VASP에 제출했지만 블록에 미포함 → `balanceOf(user) = 0`
온체인은 SUBMITTED/PENDING/FAILED 상태를 모름
그 상태를 추적하는 곳이 오프체인 원장 (mint_requests 테이블)

② 개인정보보호법 위반 — 삭제권 불가

userId "K-20240001"을 공개 체인에 저장 → 영구 기록, 삭제 불가
GDPR · 개인정보보호법 — 정보주체 삭제권(Right to be Forgotten) 보장 불가
블록체인에는 지갑 주소(0xABCD...)만. userId 연결은 오프체인 DB에만.

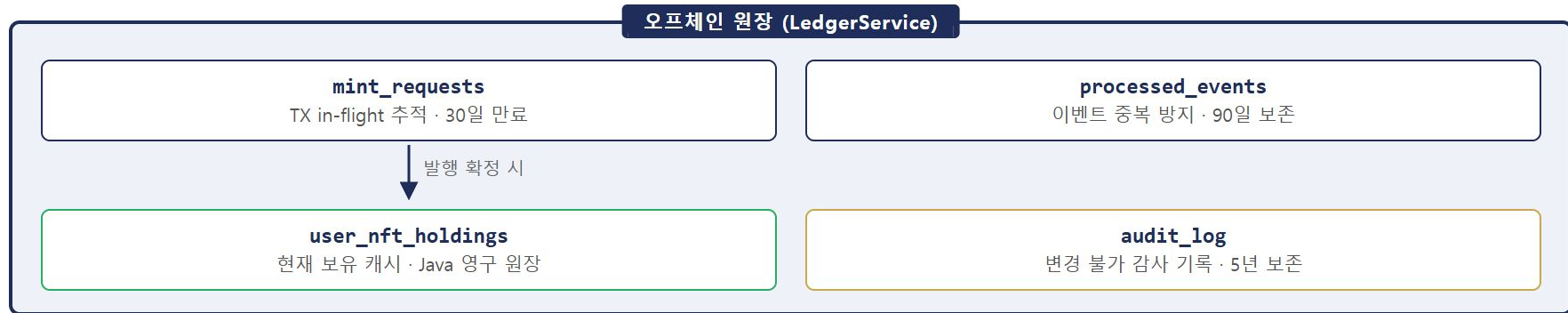
④ 동일 이벤트 중복 처리 — at-least-once

Redis Streams = **at-least-once delivery**
같은 NFTIssued 이벤트 두 번 도착 가능
"이미 처리한 이벤트 목록" → processed_events 테이블로 차단

설계 원칙: 온체인 = 단일 진실(Source of Truth). 오프체인 원장 = 온체인에서 파생된 캐시 + 보조 추적 장치. 원장이 틀렸을 때 → 온체인 기준으로 보정. 역방향(DB → 온체인) 수정은 절대 금지.

4개 테이블 역할 분담

각 테이블이 무엇을 담당하는가 — 단일 역할 원칙 (Single Responsibility)



테이블	역할	관리 주체	데이터 수명
mint_requests	TX in-flight 상태 추적 (PENDING→FINALIZED)	Node.js LedgerService	CONFIRMED/FAILED 후 30일
processed_events	NFTIssued 이벤트 최초 1회만 처리 보장	Node.js LedgerService	90일 보존 (Reorg 감지)
user_nft_holdings	현재 NFT 보유 현황 캐시	Java Gateway 영구 원장	영구 (법적 의무)
audit_log	모든 상태 변경 불변 기록 · SHA-256 체크섬	Java Gateway + AuditLogService	5년 (가상자산법)

중요: mint_requests / processed_events는 Node.js 서비스의 임시적·기술적 데이터.

user_nft_holdings / audit_log는 Java Gateway가 관리하는 영구 금융 원장. S23~S24는 Node.js 레이어만 다룬다.

① mint_requests — TX in-flight 상태 추적

발행 요청 생성 → TX 제출 → 블록 포함 → 확정까지 전 과정 상태 기록 · 30일 만료

```
-- @ mint_requests
CREATE TABLE mint_requests (
  request_id  UUID          PRIMARY KEY,
  user_id     VARCHAR(64)   NOT NULL,
  policy_id   VARCHAR(64)   NOT NULL,
  status      VARCHAR(16)   NOT NULL DEFAULT 'REQUESTED',
  tx_hash     VARCHAR(66),   -- SUBMITTED 이후
  token_id    NUMERIC,       -- CONFIRMED 이후
  block_number NUMERIC,      -- MINED 이후
  error_msg   TEXT,         -- FAILED 시
  retry_count INTEGER NOT NULL DEFAULT 0,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  updated_at  TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_mint_requests_user
  ON mint_requests(user_id);
CREATE INDEX idx_mint_requests_status
  ON mint_requests(status); -- pollStaleRequests용
```

실제 레코드 예시 (MINED 상태 — token_id 아직 NULL)

```
request_id : "a1b2c3d4-..."
user_id    : "K-20240001"
policy_id   : "WALK-10000"
status      : "MINED"
tx_hash     : "0x3f2a...8e91"
token_id    : NULL      ← 아직 확정 전이라 모름
error_msg   : NULL
created_at  : 2024-03-15 09:01:00
updated_at  : 2024-03-15 09:01:45 ← SUBMITTED 때 갱신
```

token_id가 NULL인 이유: tokenId는 온체인 이벤트(NFTIssued)를 수신해야 확정.

MINED = 블록 포함 확인이지만, 어떤 tokenId가 발급됐는지는 아직 모름.
ConsumerGroupWorker가 NFTIssued 수신 후 CONFIRMED 전이 시 채운다.

데이터 수명: CONFIRMED 또는 FAILED 확정 후 30일 자동 만료.
개인정보보호법 §21 — 처리 목적 달성 후 삭제 의무.

tokenId — 온체인 이벤트로부터 확정되는 값

발행 전엔 알 수 없다 · ERC-1155 컨트랙트가 할당 · NFTIssued 이벤트로 오프체인에 전달

발행 흐름 — tokenId가 언제 확정되나

1 IssuerService → VASP에 mintNFT 요청

파라미터: `to=0xAAA, amount=1`

tokenId 지정 없음 — 컨트랙트가 알아서 할당

2 TX 블록 포함 (MINED)

`mint_requests.token_id = NULL` ← 아직 모름

블록에 실렸다는 것만 알고, 어떤 tokenId가 발급됐는지는 모름

3 ERC-1155 컨트랙트가 NFTIssued 이벤트 emit

`NFTIssued(tokenId=1001, to=0xAAA, ...)`

이 순간 비로소 tokenId=1001 확정

4 ConsumerGroupWorker가 이벤트 수신 (CONFIRMED)

`mint_requests UPDATE SET token_id=1001, status='CONFIRMED'`

`user_nft_holdings INSERT (user_id, token_id=1001)`

왜 MINED 단계까지 NULL인가?

TX가 블록에 포함됐다는 것 = 컨트랙트 실행이 확정됐다는 것.

하지만 컨트랙트가 발급한 tokenId 값은 **체인 이벤트를 읽어야** 알 수 있다.
이벤트 수신 전까지 오프체인 원장은 tokenId를 알 방법이 없다.

tokenId 범위 — 왜 bigint인가?

EVM의 tokenId 타입은 `uint256` ($0 \sim 2^{256}-1$).

JS `number` 는 $2^{53}-1$ 까지만 정확히 표현 → 큰 tokenId에서 값 손실.

반드시 `BigInt` 사용. DB 저장 시 `.toString()`, 읽을 때

`BigInt(row['token_id'])`.

tokenId는 전역 고유값인가?

ERC-1155에서 tokenId는 컨트랙트 주소 내에서 고유.

다른 컨트랙트의 tokenId=1001과 이 컨트랙트의 tokenId=1001은 다른 NFT.

pollStaleRequests() — PENDING 장애 복구 폴링

5분 넘게 묶인 요청 감지 · VASP API 재조회 · status 인덱스가 실제로 쓰이는 곳

왜 필요한가

문제 TX를 VASP에 제출했는데 응답 없음, 서버 재시작, 네트워크 장애
→ 요청이 **PENDING** 에 영원히 갇힘

해결 5분 크론이 주기적으로 끊어서 VASP에 직접 물어봄

실행 흐름

1 DB 조회 — status 인덱스 사용

```
SELECT * FROM mint_requests
WHERE status = 'PENDING'
AND created_at < NOW() - INTERVAL '5 minutes'
```

2 건별 VASP API 재조회

```
vasp.getStatus(req.txHash)
```

3 **confirmed** → `handleConfirmed()`

failed / not_found → `handleFailed()`

pending → 아무것도 안 함 (다음 폴링 때 재확인)

```
// TxStateMachineService.ts - line 370
async pollStaleRequests(): Promise<{ processed: number }> {
  // * PENDING 5분 초과 목록 (status 인덱스 사용)
  const stale = await this.repo.findPendingOlderThan(5);
  let processed = 0;
  for (const req of stale) {
    if (!req.txHash) continue;
    // * VASP에 직접 물어봄
    const result = await this.vasp.getStatus(req.txHash);
    // * 결과별 처리
    switch (result.status) {
      case 'confirmed':
        await this.handleConfirmed(req.id); break;
      case 'failed':
        await this.handleFailed(
          req.id, result.revertReason ?? 'failed'); break;
      case 'not_found':
        await this.handleFailed(
          req.id, 'tx not found in mempool'); break;
      // 'pending' → 아무것도 안 함
    }
    processed++;
  }
  return { processed };
}
```

크론 = **threshold** = 5분. 첫 번째 폴링에서 바로 감지. **status** 인덱스 없으면 매 폴링마다 폴스캔.

② processed_events — 이벤트 중복 방지 장치

at-least-once delivery에서 같은 이벤트를 단 1회만 처리 · UNIQUE(tx_hash, log_index)가 핵심

```
-- @ processed_events
CREATE TABLE processed_events (
  id          SERIAL          PRIMARY KEY,
  tx_hash     VARCHAR(66)     NOT NULL,
  log_index   INTEGER         NOT NULL,
  event_name  VARCHAR(64)     NOT NULL,
  block_number NUMERIC        NOT NULL,
  payload     JSONB,
  processed_at TIMESTAMPTZ    DEFAULT NOW(),
  UNIQUE (tx_hash, log_index) -- 이 줄이 전부
);
CREATE INDEX idx_processed_events_block
ON processed_events(block_number);
```

왜 tx_hash만으로 안 되나?

TX 0x3f2a 안에: log[0]=NFTIssued(tokenId=1001, 0xAAA) ·

log[1]=NFTIssued(tokenId=1002, 0xBBB)

tx_hash만 UNIQUE면 log[1]이 "이미 처리됨"으로 잘못 차단 → 0xBBB는 NFT 못 받음.

log_index가 한 TX 안에서 각 이벤트를 구분하는 유일한 키.

실제 레코드 예시

```
id          : 1
tx_hash     : "0x3f2a...8e91"
log_index   : 0
event_name  : "NFTIssued"
block_number : 12345
payload     : {"tokenId": 1001, "to": "0xAAA..."}
processed_at : 2024-03-15 09:01:50
```

2번째 같은 (tx_hash=0x3f2a, log_index=0)로 INSERT 시도
→ ON CONFLICT DO NOTHING → rows.length==0
→ { skipped: true } → 상위 호출부 즉시 return

✓ DB UNIQUE 제약이 단일 원자 연산으로 처리
Race Condition 없음 — SELECT-then-INSERT보다 안전

데이터 수명: 90일 보존. Reorg 감지를 위해 최근 블록 이벤트 유지 필요.

log_index — Etherscan에서 직접 확인하기

하나의 TX 안에서 이벤트마다 부여되는 순번 · (tx_hash, log_index) 쌍이 UNIQUE KEY의 근거

 Etherscan Transaction Receipt Event Logs

왼쪽 번호 = log_index

log_index	이벤트
158	Transfer(from, to, value)
159	Approval(owner, spender, value)
160	Wrapped Ether(src, dst, wad)

왜 tx_hash만으로는 안 되나?

하나의 TX에서 이벤트 3개 발생.

tx_hash가 같으면 158·159·160을 구분 불가.

→ (tx_hash, log_index) 쌍이 필요

블록 기준 vs TX 기준

Etherscan의 번호(158~)는 블록 전체 기준.

코드에서는 TX 내 순번(0, 1, 2...)을 쓰기도 함.

어느 쪽이든 tx_hash 가 기준점이 됨.

왜 TX 단위가 아니라 이벤트 단위로 관리하는가

TX는 이벤트들의 묶음 · 비즈니스 액션의 단위 = 개별 이벤트 · 부분 실패 복구가 핵심

TX 하나에 이벤트 여러 개

TX 0x3f2a 안에:

```
log[0]: NFTIssued(tokenId=1001, to=0xAAA) ← 사용자 A  
log[1]: NFTIssued(tokenId=1002, to=0xBBB) ← 사용자 B  
log[2]: NFTIssued(tokenId=1003, to=0xCCC) ← 사용자 C
```

TX 단위로만 기록하면

- 1 log[0] 처리 → 사용자 A 원장 업데이트
- 2 log[1] 처리 → 사용자 B 원장 업데이트
-  서버 다운
-  문제 재시작 → "0x3f2a 이미 처리됨" → skip
→ 사용자 C NFT 미발급

이벤트 단위로 기록하면

- 1 log[0] 처리 → (0x3f2a, 0) 기록
- 2 log[1] 처리 → (0x3f2a, 1) 기록
-  서버 다운
- 3 재시작 → log[0], log[1] → ON CONFLICT skip
log[2]만 처리 → 사용자 C 정상 수령

이벤트 = 독립적인 비즈니스 액션

NFTIssued 1개 = 사용자 1명의 NFT 발급.

멥등성 보장 단위도 이벤트여야 한다.

블록체인 TX 원자성 ≠ 오프체인 처리 원자성

블록체인에서는 원자적 · 우리 **Consumer** 처리 과정에서 원자성이 깨진다

블록체인 레벨 — 원자적

체인 TX 0x3f2a 안의 log[0], log[1], log[2]는
모두 성공하거나 모두 실패 — 절반만 되는 경우 없음

오프체인 처리 — 원자적 아님

블록체인	Consumer 처리
TX 0x3f2a	이벤트 하나씩 꺼냄
log[0] → DB 반영 완료 ✓	
log[1] → DB 반영 완료 ✓	
log[2] → 처리 중 🚨 서버 다운	

→ 체인에는 3개 모두 존재
→ DB에는 2개만 반영된 상태

왜 TX 단위 처리가 어려운가

블록체인

- 이벤트 감지
- Redis Stream ← 이벤트를 하나씩 넣음
- Consumer ← 이벤트를 하나씩 꺼내서 처리
 - log[0] 처리 → ack
 - log[1] 처리 → ack ← 그 사이 죽을 수 있음
 - log[2] 처리 → ack

TX 단위로 묶어서 처리하면

이벤트 타입별 Consumer 분리 불가

병렬 처리 불가 · 유연성 상실

결론: 원자성이 깨지는 건 블록체인이 아니라

우리 **Consumer**가 이벤트를 소화하는 과정.

그래서 이벤트 단위 멍등성이 필요하다.

ON CONFLICT DO NOTHING + RETURNING id

단일 원자 연산으로 중복 감지 · SELECT-then-INSERT의 Race Condition 없음

RETURNING이 "중복 감지 신호"가 되는 원리

신규 충돌 없음 → INSERT 성공 → RETURNING이 삽입된 행 반환

`rows = [{ id: 42 }] → rows.length === 1 → 계속 진행`

중복 충돌 있음 → INSERT 조용히 무시 → RETURNING도 아무것도 안 돌려줌

`rows = [] → rows.length === 0 → { skipped: true } 반환`

왜 SELECT-then-INSERT 안 쓰나

[프로세스 A] SELECT → 없음 확인
[프로세스 B] SELECT → 없음 확인 ← 동시에 실행
[프로세스 A] INSERT → 성공
[프로세스 B] INSERT → 성공 ← 중복 발행!

ON CONFLICT DO NOTHING은 DB가 단일 원자 연산 처리 → 한 쪽만 성공 보장

```
-- 일반 INSERT + RETURNING
INSERT INTO processed_events (tx_hash, log_index, ...)
VALUES ('0x3f2a', 0, ...)
RETURNING id;

-- → [{ id: 42 }] 항상 삽입된 행 반환

-- ON CONFLICT DO NOTHING + RETURNING
INSERT INTO processed_events (tx_hash, log_index, ...)
VALUES ('0x3f2a', 0, ...)
ON CONFLICT (tx_hash, log_index) DO NOTHING
RETURNING id;

-- 첫 번째: → [{ id: 42 }] rows.length === 1
-- 두 번째: → [] rows.length === 0
```

```
const rows = await db.query(`
  INSERT INTO processed_events ...
  ON CONFLICT DO NOTHING
  RETURNING id
`, [...]);

if (rows.length === 0)
  return { skipped: true }; // 중복 → 즉시 return

// rows.length === 1 → 신규 → 계속 진행
```

payload — NFTIssued 이벤트의 원본 데이터

블록체인이 우리에게 준 이벤트 파라미터 저장 · MINT_REQUEST가 아님

payload는 무엇인가

- 아님** MINT_REQUEST가 아니다
mint_requests는 별도 테이블에 있고, processed_events는 그것과 무관하다
- 맞음** 온체인 이벤트(NFTIssued)의 파라미터
블록체인이 emit한 이벤트 로그의 args를 그대로 저장

두 테이블 방향 비교

테이블	방향	역할
mint_requests	우리 → 체인	보낸 TX 추적
processed_events	체인 → 우리	받은 이벤트 중복 방지

payload는 **JSONB nullable**. 파라미터 없는 단순 이벤트도 있을 수 있어서.

NFTIssued 이벤트 흐름

- 스마트 컨트랙트가 NFTIssued 이벤트 emit
`tokenId=1001, to=0xAAA...`
- 이벤트 리스너가 수신 → payload로 파싱
- processed_events에 INSERT
payload 컬럼에 이벤트 args 저장

```
-- 실제 저장되는 payload 값
-- NFTIssued(tokenId, to) 이벤트 args
{
  "tokenId": 1001,
  "to": "0xAAA..."
}
```

```
-- 나중에 재처리·감사 추적 시 원본 확인용
-- 처리 당시 이벤트 내용을 영구 보존
```

③ user_nft_holdings — 현재 NFT 보유 현황 캐시

앱 "내 NFT" 목록의 실제 데이터 소스 · 온체인 RPC 없이 5ms 응답 · Java Gateway 영구 원장

```
-- @ user_nft_holdings (Phase 1 확정)
CREATE TABLE user_nft_holdings (
  id          BIGSERIAL PRIMARY KEY,
  user_id     VARCHAR(64) NOT NULL,
  token_id    BIGINT     NOT NULL,
  contract_addr VARCHAR(42) NOT NULL,
  chain_id    INT        NOT NULL,
  amount      BIGINT     NOT NULL DEFAULT 1,
  acquired_at TIMESTAMPTZ NOT NULL,
  released_at TIMESTAMPTZ,
  on_chain_tx  VARCHAR(66) NOT NULL,
  UNIQUE(user_id, token_id, contract_addr, chain_id)
);
CREATE INDEX idx_holdings_user
  ON user_nft_holdings(user_id);
```

읽기 경로 (온체인 호출 없음):

```
SELECT * FROM user_nft_holdings WHERE user_id='K-20240001'
```

→ 인덱스 스캔 → 5ms vs. 온체인 balanceOf() RPC → 200~500ms

데이터 수명: 영구 보존 (법적 의무).

Java Gateway 영구 원장. ReconcileService(S25)가 주기적으로 온체인과 비교.

실제 레코드 예시 (발행 확정 후 삽입)

```
id          : 1
user_id     : "K-20240001"
token_id    : 1001
contract_addr : "0x4b0897b0..."
chain_id    : 1
amount      : 3
acquired_at  : 2024-03-15 09:01:50
on_chain_tx  : "0xaaaa0000..."
released_at  : null
```

UNIQUE(user_id, token_id, contract_addr, chain_id)의 두 가지 역할:

- ① 비즈니스 규칙 — 같은 사람이 같은 (컨트랙트·tokenId)를 중복 보유 불가
- ② at-least-once 방어 — 재처리 시에도 중복 INSERT 자동 차단
mint_requests/processed_events가 뚫려도 이 제약이 최종 방어선.

쓰기: on-chain NFTIssued 이벤트 수신 후 Java Gateway 호출

```
INSERT ... ON CONFLICT (user_id, token_id, contract_addr, chain_id) DO
NOTHING
```


user_nft_holdings — Phase 2/3 대응 설계

소각/양도 상태 추적 · DELETE 없는 감사 보존 · amount/chain_id/contract_addr는 Phase 1에 포함

Phase 2/3 추가 컬럼

문제 추가 컬럼

소각/양도 후 row DELETE → 기록 소멸

status VARCHAR(16)

상태 변경 시각 추적 불가

updated_at TIMESTAMPTZ

Phase 1에서 이미 확정

amount · chain_id · contract_addr · on_chain_tx

UNIQUE(user_id, token_id, contract_addr, chain_id) — 이미 적용

status 값: ACTIVE 보유 중 · BURNED 소각 · TRANSFERRED 양도

소각/양도 시 DELETE 하지 않고 status 변경 → 감사 기록 보존

-- Phase 2/3: Phase 1 베이스에 추가

```
ALTER TABLE user_nft_holdings
  ADD COLUMN status VARCHAR(16)
    NOT NULL DEFAULT 'ACTIVE',
  -- ACTIVE / BURNED / TRANSFERRED
  ADD COLUMN updated_at TIMESTAMPTZ
    NOT NULL DEFAULT NOW();
```

-- Phase 1 베이스 (참고)

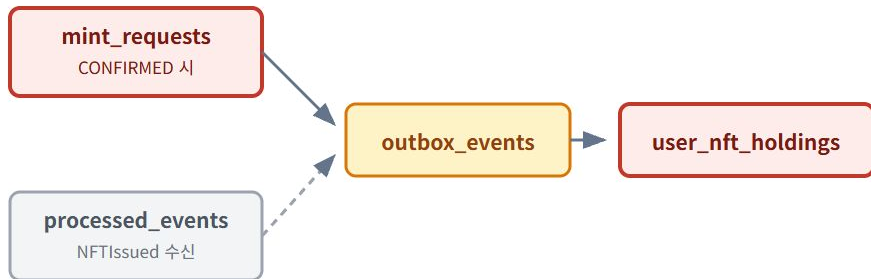
```
-- user_id, token_id, contract_addr
-- chain_id, amount, acquired_at
-- released_at, on_chain_tx
-- UNIQUE(user_id, token_id,
--         contract_addr, chain_id)
```

```
CREATE INDEX idx_holdings_status
  ON user_nft_holdings(status);
CREATE INDEX idx_holdings_chain
  ON user_nft_holdings(chain_id, contract_addr);
```

Phase 2/3 확장 시 연관 테이블 변경 범위

user_nft_holdings 혼자 바뀌서는 안 됨 · mint_requests 연결 고리가 핵심

테이블 간 흐름



필수 변경 — 2개 테이블

mint_requests chain_id, contract_addr 추가 CONFIRMED → holding 연결 고리

user_nft_holdings status, updated_at 추가 amount · chain_id 등은 Phase 1에 포함

✓ processed_events — JSONB payload가 신규 필드 유연 수용, 스키마 변경 불필요

테이블별 변경 범위

테이블	변경	이유
mint_requests	필수	chain_id, contract_addr 추가 CONFIRMED → holding 생성 연결
user_nft_holdings	필수	status, updated_at 추가 (amount · chain_id 등은 Phase 1에 포함)
outbox_events	페이로드만	JSONB payload에 status 필드 추가
processed_events	불필요	JSONB payload로 유연하게 수용
audit_log	불필요	before/after_state JSONB — 그대로

mint_requests 마이그레이션

```
ALTER TABLE mint_requests
ADD COLUMN chain_id VARCHAR(32)
NOT NULL DEFAULT 'ethereum-mainnet',
ADD COLUMN contract_addr VARCHAR(42)
NOT NULL DEFAULT '0x...';
```

④ audit_log — 변경 불가 감사 기록

전자금융감독규정 §34 (접근기록 1년+) · 가상자산이용자보호법 §15 (거래기록 5년) · INSERT-only

```
-- @ audit_log - INSERT-only
CREATE TABLE audit_log (
  id          BIGSERIAL    PRIMARY KEY,
  event_time  TIMESTAMPTZ NOT NULL,
  actor       VARCHAR(64)  NOT NULL,
  action      VARCHAR(64)  NOT NULL,
  resource_type VARCHAR(32) NOT NULL,
  resource_id VARCHAR(128) NOT NULL,
  before_state JSONB,
  after_state JSONB        NOT NULL,
  prev_checksum VARCHAR(64), -- NULL = genesis
  checksum      VARCHAR(64) NOT NULL
  -- SHA-256(prevChecksum+eventTime+actor+action+resourceId+afterState)
);
CREATE INDEX idx_audit_resource
  ON audit_log(resource_type, resource_id);
CREATE INDEX idx_audit_actor ON audit_log(actor);
```

실제 레코드 예시 (발행 요청 생성 시점)

```
actor       : "system:IssuerService"
action      : "MINT_REQUESTED"
resource_type : "mint_request"
resource_id  : "a1b2c3d4-..."
before_state : NULL
after_state  : {"status":"REQUESTED","userId":"K-20240001"}
prev_checksum : "0000...0"
checksum     : "e3b0c44298fc..."
```

before_state: 최초 생성 시 NULL · 상태 변경 시 변경 전 스냅샷

INSERT-only 원칙: 레코드 수정·삭제 절대 금지. 보정은 새 레코드 추가.
위반 시 전자금융감독규정 §34 · 가상자산이용자보호법 §15 위반.

Hash chain checksum:

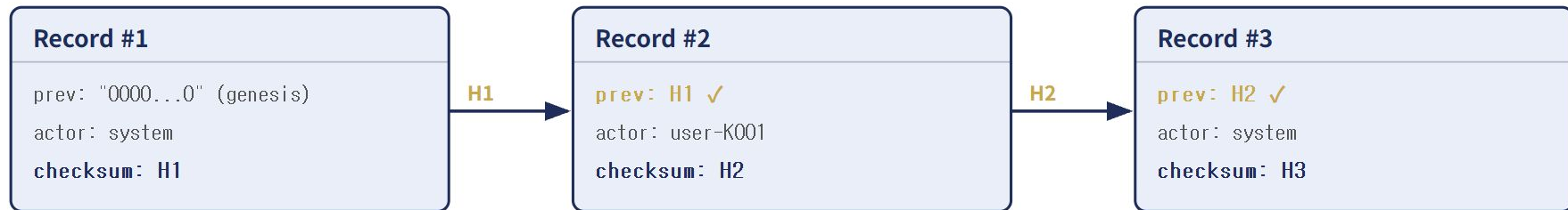
```
SHA-256(prevChecksum + eventTime + actor + action + resourceId +
afterState)
```

수정 후 checksum 재계산해도 다음 레코드 prevChecksum과 불일치 → 탐지
첫 레코드 prevChecksum = "0000...0" (genesis) · S26에서 상세히 다룬다.

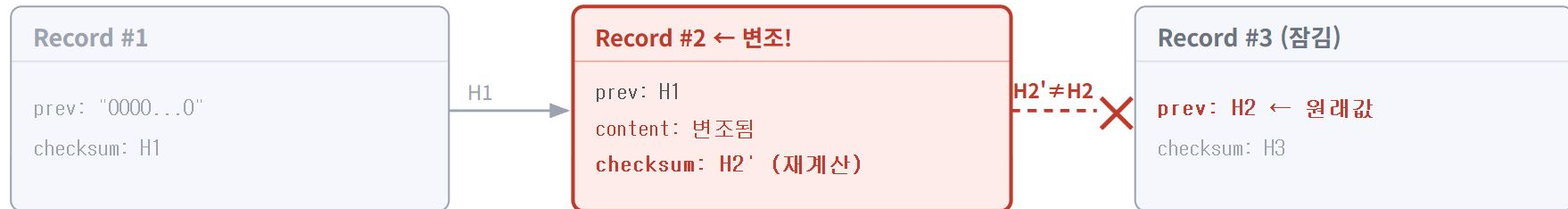
Hash Chain — 레코드 연쇄 무결성 보장

단순 checksum은 재계산으로 우회 가능 · prevChecksum을 포함해 체인 전체를 보호

정상 체인



변조 시나리오 — Record #2 내용 수정 후 checksum 재계산



$\text{prev}(\text{H2}) \neq \text{checksum}[2](\text{H2}') \rightarrow \text{X}$ 감지

LedgerService — 코드 아키텍처와 의존성 구조

실제 파일 위치 · 의존성 역전(DI) 패턴 · 왜 인터페이스를 주입하는가

파일 위치

internal/packages/core-banking/src/ledger/LedgerService.ts

LedgerService

constructor()

db: DatabaseClient (인터페이스)

auditLog: AuditLogClient (인터페이스)

createMintRequest()

getMintRequest()

updateMintRequest()

상태 전이 가드 포함

recordProcessedEvent()

ON CONFLICT DO NOTHING

의존성 역전 원칙(DIP):

LedgerService는 PostgreSQL 구체 클래스를 모른다.

DatabaseClient 인터페이스에만 의존 → 테스트 시 MockDB 주입 가능

```
interface DatabaseClient {
  query(
    sql: string,
    params?: unknown[],
  ): Promise<{ rows: Record<string, unknown>[] }>;
}
```

```
interface AuditLogClient {
  log(entry: {
    actor:      string;
    action:     string;
    resourceType: string;
    resourceId: string;
    beforeState?: unknown;
    afterState:  unknown;
  }): Promise<void>;
}
```

raw SQL vs ORM: DatabaseClient는 raw SQL 직접 실행.

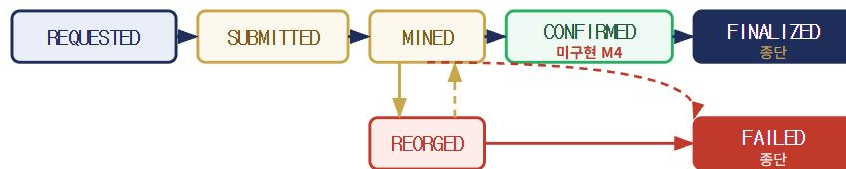
ORM을 쓰지 않는 이유 → ON CONFLICT DO NOTHING, RETURNING id 같은 PostgreSQL 전용 문법을 그대로 활용해야 하기 때문.

MintStatus 타입 7종 + VALID_TRANSITIONS 맵

LedgerService가 관리하는 상태 전이 — 허용된 경로만 통과

```
export type MintStatus =  
  | 'REQUESTED' // 발행 요청 생성 (초기 상태)  
  | 'SUBMITTED' // VASP에 TX 제출 완료  
  | 'MINED' // 블록 포함 - Reorg 가능  
  | 'CONFIRMED' // 내부 원장 반영 완료 (종단)  
  | 'FINALIZED' // PoS Finality 확보 (종단)  
  | 'FAILED' // 발행 실패 (종단)  
  | 'REORGED'; // 체인 재편으로 TX 소실  
  
private static readonly VALID_TRANSITIONS:  
  Record<MintStatus, MintStatus[]> = {  
    REQUESTED: ['SUBMITTED', 'FAILED'],  
    SUBMITTED: ['MINED', 'FAILED'],  
    MINED: ['CONFIRMED', 'REORGED', 'FAILED'],  
    CONFIRMED: ['FINALIZED'],  
    FINALIZED: [], // 종단 - 원장 업데이트 완료  
    FAILED: [], // 종단  
    REORGED: ['MINED', 'FAILED'],  
  };
```

상태 전이 다이어그램:



* REQUESTED · SUBMITTED → FAILED 직접 전이 가능

상태	txHash	설정 주체
REQUESTED	NULL	LedgerService.createMintRequest()
SUBMITTED	있음 (필수)	TxStateMachineService
MINED	있음	TxStateMachineService
CONFIRMED	있음 (필수)	ConsumerGroupWorker 미구현 — M4
FINALIZED	있음	TxStateMachineService
REORGED	있음 (무효)	TxStateMachineService
FAILED	있을 수도	TxStateMachineService

MintRequest 인터페이스 + createMintRequest() 구현

요청 생성 시 무엇이 INSERT되고 AuditLog에 무엇이 기록되는가

MintRequest 인터페이스 필드

필드	타입	설명
id	string	UUID — randomUUID() 생성
userId / policyId	string	요청자 · 정책 식별자
status	MintStatus	현재 상태 (7종)
txHash?	string	SUBMITTED 이후 설정
tokenId?	bigint	CONFIRMED 이후 설정
errorMsg?	string	FAILED 시 원인 메시지

bigint 이유: EVM tokenId는 uint256 (최대 $2^{256}-1$). JS `number` 는 $2^{53}-1$ 초과 시 정밀도 손실 → `bigint` 필수.

createMintRequest() 처리 흐름

- 1 **UUID 생성** — `randomUUID()` (Node.js crypto 모듈). 호출자가 아닌 서비스가 직접 생성.
- 2 **DB INSERT** — `mint_requests` 에 `status='PENDING'` 으로 삽입. `created_at` 과 `updated_at` 에 동일 타임스탬프(`$4,$4`).
- 3 **감사 기록** — `auditLog.log()` 호출. 최초 생성이므로 `beforeState` 없음, `afterState` 만 기록.
- 4 **객체 반환** — DB 재조회 없이 INSERT에 사용한 값으로 바로 반환. DB 왕복 1회 절감.

auditLog.log() 패턴: DB INSERT와 감사 기록을 항상 함께 실행. 어느 하나라도 빠지면 규제 위반.

updateMintRequest() — 상태 전이 가드 구현

허용되지 않은 전이를 DB 업데이트 전에 차단 · beforeState/afterState 감사 기록

처리 4단계

- 1 현재 상태 조회 — `getMintRequest(requestId)` 호출. 없으면 `MintRequestNotFoundError` throw.
- 2 전이 허용 체크 — `VALID_TRANSITIONS[current.status]` 에서 허용 목록 조회
→ `includes(patch.status)` false면 `InvalidStateTransitionError` throw.
- 3 DB UPDATE — `status, tx_hash, token_id, error_msg, updated_at` 갱신.
`tokenId`는 `bigint` → `string` 변환 후 저장.
- 4 감사 기록 — `beforeState: current` (DB 조회값) · `afterState: {...current, ...patch}` . 변경 전/후 모두 기록.

설계 포인트

VALID_TRANSITIONS이 규칙을 소유

- 상태 전이 허용 여부는 코드 로직이 아닌 맵이 결정
- 새 상태 추가 시 맵만 수정 — 함수 로직은 불변
- 테스트 시 맵을 직접 검증 가능

beforeState를 DB 조회로 확보

- Step 1에서 조회한 `current`를 `beforeState`로 그대로 사용
- 호출자가 이전 상태를 넘겨줄 필요 없음 — 원장이 직접 확인

FAILED → CONFIRMED 시도:

`VALID_TRANSITIONS[FAILED] = []` → `includes('CONFIRMED') = false`
→ `InvalidStateTransitionError` throw

getMintRequest() + 에러 클래스 설계

DB row → TypeScript 객체 변환 · MintRequestNotFoundError · InvalidStateTransitionError

getMintRequest() 핵심 동작

조회 & null 반환

- `SELECT * FROM mint_requests WHERE id = $1`
- `rows.length === 0`이면 **null** 반환 — 예외가 아님
- 존재 여부를 호출자가 직접 판단하도록 설계

DB row → TypeScript 객체 매핑

- DB `user_id` → TS `userId` (snake → camelCase)
- `tokenId` : DB numeric → string → `BigInt()` 변환 필수
- `createdAt` / `updatedAt` : string → `new Date()`
- optional 필드(`txHash`, `tokenId`, `errorMsg`)는 undefined 처리

BigInt 이유: DB는 numeric을 string으로 반환. JS `number` 는 $2^{53}-1$ 초과 시 정밀도 손실 → `BigInt()` 필수.

에러 클래스 설계

클래스	발생 조건	HTTP
<code>MintRequestNotFoundError</code>	id 조회 시 없음	404
<code>InvalidStateTransitionError</code>	허용되지 않은 전이	409

Error를 extends하는 이유

- `instanceof MintRequestNotFoundError` — catch 블록에서 타입 구분
- 에러별 HTTP 상태코드 분기 처리 가능
- `this.name` 지정 → 스택 트레이스에 클래스명 표시

Error 상속 주의: `super(message)` 호출 후 반드시 `this.name` 재지정. JS에서 Error subclass는 name이 자동 설정되지 않음.

쓰기 경로 1/6 — 발행 요청 생성

걷기 목표 달성 이벤트 수신 · IssuerService → createMintRequest()

① 발행요청

② TX제출

③ MINED

④ FINALIZED

⑤ CONFIRMED

⑥ 완료

항목 내용

호출 시점 사용자 걷기 목표 달성 이벤트 수신

호출 주체 IssuerService

호출 함수 `createMintRequest(userId, policyId)`

처리 순서:

- ① `randomUUID()` → `requestId` 생성
- ② `mint_requests` INSERT (`status='PENDING'`)
- ③ `auditLog.log({ action: 'MINT_REQUESTED' })`

테이블 상태 변화

mint_requests

- 신규 레코드 INSERT
- `status` = **PENDING**
- `tx_hash`, `token_id` = `null`

audit_log

- `action`: **MINT_REQUESTED**
- `beforeState`: 없음 (최초 생성)
- `afterState`: { `userId`, `policyId`, `status: 'PENDING'` }

쓰기 경로 2/6 — TX 제출 (SUBMITTED)

VASP TX 제출 직후 · txHash 확보 · mint_requests에 tx_hash 기록

① 발행요청

② TX제출

③ MINED

④ FINALIZED

⑤ CONFIRMED

⑥ 완료

항목 내용

호출 시점 VASP TX 제출 완료, txHash 수신 직후

호출 주체 TxStateMachineService

호출 함수 `updateMintRequest({ status: 'SUBMITTED', txHash })`

txHash 필수: SUBMITTED 전이 시 txHash 누락이면 TX 추적 불가 → 누락 시 FAILED 처리 후 재발행.

테이블 상태 변화

mint_requests

- **PENDING** → **SUBMITTED**
- tx_hash = '0x3f2a...' (이 단계에서 처음 설정)
- updated_at 갱신

audit_log

- action: **STATUS_SUBMITTED**
- beforeState: { status: 'PENDING' }
- afterState: { status: 'SUBMITTED', txHash }

쓰기 경로 3/6 — 블록 포함 확인 (MINED)

TX가 블록에 포함됨 감지 · 아직 Finalized 아님 — Reorg 가능성 있음

① 발행요청

② TX제출

③ MINED

④ FINALIZED

⑤ CONFIRMED

⑥ 완료

항목 내용

호출 시점 TX가 블록에 포함됨 확인 (콜백/풀링)

호출 주체 TxStateMachineService

호출 함수 `updateMintRequest({ status: 'MINED' })`

Reorg 경계: MINED는 블록 포함만 확인. 체인 재구성(Reorg) 발생 시
REORGED 전이 가능. FINALIZED 전까지는 최종 확정 아님.

테이블 상태 변화

mint_requests

- **SUBMITTED** → **MINED**
- updated_at 갱신

audit_log

- action: **STATUS_MINED**
- beforeState: { status: 'SUBMITTED' }
- afterState: { status: 'MINED' }

쓰기 경로 4/6 — Finality 확보 (FINALIZED) + Java 원장 기록

PoS Finality 확보 · Java 영구 원장에 NFT 보유 기록 후 상태 전이

① 발행요청

② TX제출

③ MINED

④ FINALIZED

⑤ CONFIRMED

⑥ 완료

항목 내용

호출 시점 PoS Finality 확보 (32 epoch 이상)

호출 주체 TxStateMachineService

호출 함수 `updateMintRequest({ status: 'FINALIZED' })`
`ICoreBankingAdapter.recordNftHolding()`

Java 원장 기록 순서: recordNftHolding() 완료 후 FINALIZED 전이. Java 원장 기록 실패 시 재시도 — 전이하지 않음.

테이블 상태 변화

mint_requests

- MINED → FINALIZED
- updated_at 갱신

audit_log

- action: STATUS_FINALIZED

Java 영구 원장 (외부)

- recordNftHolding(userId, tokenId, policyId)
- user_nft_holdings 업데이트 (Java Gateway 소유)

쓰기 경로 5/6 — NFTIssued 이벤트 처리 (CONFIRMED)

ConsumerGroupWorker · 먹등성 확인 후 상태 확정 · tokenId 최초 기록

① 발행요청

② TX제출

③ MINED

④ FINALIZED

⑤ CONFIRMED

⑥ 완료

항목 내용

호출 시점 NFTIssued 이벤트 수신 (kyobo:events 스트림)

호출 주체 **ConsumerGroupWorker**

호출 함수
① `recordProcessedEvent(txHash, logIndex)`
② `updateMintRequest({ status: 'CONFIRMED', tokenId })`

순서 불변: recordProcessedEvent() 반드시 첫 번째.
skipped=true이면 즉시 return — updateMintRequest() 호출 금지.

테이블 상태 변화

processed_events

- 신규: INSERT (tx_hash, log_index, 'NFTIssued')
- 중복: ON CONFLICT DO NOTHING → skipped=true → return

mint_requests

- **FINALIZED** → **CONFIRMED**
- token_id = 이 단계에서 처음 확정

audit_log

- action: `STATUS_CONFIRMED`

쓰기 경로 6/6 — 완료 상태 요약

모든 단계 완료 후 테이블 최종 상태 · 읽기 경로 확인

① 발행요청

② TX제출

③ MINED

④ FINALIZED

⑤ CONFIRMED

⑥ 완료

테이블 최종 상태

테이블	최종 상태
mint_requests	status = CONFIRMED , token_id 확정
processed_events	NFTIssued 이벤트 레코드 존재 (재처리 방지)
audit_log	5건: REQUESTED → SUBMITTED → MINED → FINALIZED → CONFIRMED
user_nft_holdings	Java Gateway 소유 — NFT 보유 기록 완료

읽기 경로

앱 "내 NFT" 조회

- Java Gateway → `user_nft_holdings` 조회
- 응답 ~5ms (Node.js LedgerService 불필요)

발행 상태 조회

- `getMintRequest(requestId)` → status + token_id 반환

mint_requests는 운영 데이터: 발행 추적·재시도·감사용. 앱 조회에서 직접 노출 금지.

ON CONFLICT DO NOTHING — 이벤트 멍등성 패턴

at-least-once 재전달 환경에서 중복 처리를 막는 유일하게 안전한 방법

핵심 패턴 — recordProcessedEvent()

```
INSERT INTO processed_events
(tx_hash, log_index, event_name, block_number, payload)
VALUES ($1, $2, $3, $4, $5)
ON CONFLICT (tx_hash, log_index) DO NOTHING
RETURNING id;
```

```
const rows = await db.query(sql, [txHash, logIndex, ...]);

if (rows.length === 0) {
  return { skipped: true }; // 중복 → 즉시 종료
}
return { skipped: false }; // 신규 → 계속 처리
```

RETURNING id 핵심: 신규면 id 반환, 중복이면 아무 행도 반환 안 함 → rows.length로 판별

왜 SELECT-then-INSERT는 안 되나

```
// Worker-A: SELECT → 없음 → INSERT 준비...
// Worker-B: SELECT → 없음 → INSERT 준비...
// Worker-A: INSERT 완료
// Worker-B: INSERT 완료 ← 중복 발행!
```

역방향 수정 절대 금지 · 보정 올바른 프로세스

원장은 캐시 — 온체인이 원본 · 불일치 발생 시 새 TX로만 해결

역방향 수정 절대 금지

왜 금지인가

- 원장은 캐시 — 온체인 상태가 원본
- 캐시만 수정 → 다음 Reconcile 때 또 불일치 감지 (증상 은폐)
- 원장→온체인 수정 코드 존재 시 DB 조작만으로 임의 NFT 발행 가능 → 보안 취약점

절대 하면 안 되는 것

- mint_requests status를 직접 UPDATE
- token_id를 임의 값으로 패치
- processed_events 레코드 삭제 (재처리 유도)
- audit_log 레코드 수정·삭제

보정이 필요할 때 — 올바른 프로세스

- 1 **ReconcileService가 불일치 감지** — 온체인 vs 원장 비교. 감지만, 수정 없음.
- 2 **담당자 알림 발송** — Slack/이메일 통지 → 원인 분석 시작.
- 3 **원인 분석** — TX 실패 원인, 이벤트 누락, Reorg 여부 파악.
- 4 **IssuerService 정상 프로세스로 새 TX 발행** — 기존 레코드 수정 X. 새 mint_request 생성 + 새 TX 제출.

ReconcileService는 감지 + 알림만. 수정 권한 없음.

InternalLedgerBalance — 4단계 잔액 모델 (Phase 3 예고)

Phase 1(현재)은 CONFIRMED 시 Java 원장에 위임 · Phase 3에서 직접 Custody 전환 시 활성화

```
export interface InternalLedgerBalance {  
  userId:    string;  
  tokenId:   bigint;  
  available: bigint; // = Settled - Reserved - Pending  
  reserved:  bigint; // 출금 승인됐으나 온체인 미확정  
  pending:   bigint; // TX 전송됨, 블록 미포함  
  settled:   bigint; // 온체인 Finalized 기준 최종  
  updatedAt: Date;  
}
```

```
// Phase 3에서 추가될 메서드들:  
// reserve(userId, tokenId, amount)  
//   → Available → Reserved (출금 승인 시)  
// settle(userId, tokenId, amount)  
//   → Reserved+Pending → Settled (FINALIZED 시)  
// release(userId, tokenId, amount)  
//   → Reserved → Available (출금 취소 시)  
// getBalance(userId, tokenId)  
//   → InternalLedgerBalance 반환
```

Phase 1 (현재): NFT 발행(단방향) 위주

CONFIRMED 시 Java Gateway의 영구 원장에 위임

→ RESERVE 필요성 낮음. InternalLedgerBalance 미사용.

4단계 잔액 흐름 (Phase 3):



데이터 수명 관리 + 규제 보존 의무

테이블별 데이터 보존 기간 · 자동 만료 구현 · 법적 의무와의 매핑

LedgerService 주식 (실제 코드):

- 30 mint_requests** — **CONFIRMED/FAILED** 후 30일 자동 만료
이유: 운영 편의 (종료된 요청 쌓이면 DB 부담)
- 90 processed_events** — **90일** 보존
이유: Reorg 감지 목적 (최대 Reorg 길이 << 90일 분량의 블록)

```
-- 자동 만료 구현 (PostgreSQL pg_cron)
DELETE FROM mint_requests
WHERE status IN ('CONFIRMED', 'FAILED')
  AND updated_at < NOW() - INTERVAL '30 days';

DELETE FROM processed_events
WHERE processed_at < NOW() - INTERVAL '90 days';
```

규제 보존 의무

법령	의무	테이블	기간
가상자산이용자보호법 §15	거래 기록 보존	audit_log	5년
전자금융감독규정 §34	접근 기록 보존	audit_log	1년 이상
개인정보보호법 §21	목적 달성 후 삭제	mint_requests	30일
내부 정책	Reorg 감지	processed_events	90일

감독원 조회 요건: audit_log는 즉시 조회 가능해야 함.

resource_type, actor 인덱스로 빠른 조회 보장.

INSERT-only + RLS 정책으로 변조 불가 (S26에서 상세).

실습

Docker 개발 환경 설정 WSL2 + Rancher Desktop

Step 1: WSL2 + Rancher Desktop 설치

Step 2: 환경 변수 설정 + 컨테이너 실행

Step 3: 스키마 초기화 확인

직접 실행

따라하기

README 10절 기준



실습 Step 1 — WSL2 + Rancher Desktop 설치

Windows 환경에서 Docker를 실행하기 위한 기반 설치

WSL2 설치

- 1 PowerShell (관리자) 열기
- 2 `wsl --install` 실행 → 재부팅
- 3 Ubuntu 창 열리면 UNIX username / password 설정
- 4 `wsl --list --verbose` 로 STATE=Running 확인

주의: 재부팅 후 Ubuntu 창이 자동으로 열립니다. 닫지 말고 사용자 설정까지 완료하세요.

Rancher Desktop 설치

- 1 rancherdesktop.io 에서 Windows installer 다운로드 후 실행
- 2 설치 완료 후 첫 실행 → **Container Engine: dockerd (moby)** 선택
- 3 System tray 아이콘이 **Running** 상태가 될 때까지 대기 (1~2분)
- 4 `docker info` 실행 → Server Version 출력되면 완료

containerd → moby 변경: Preferences → Container Engine → dockerd (moby) 로 변경 후 Apply 클릭

실습 Step 2 — .env 파일 생성

파일 위치: `infrastructure/docker/.env`

- 1 프로젝트 루트에서 `infrastructure/docker/` 디렉터리로 이동
- 2 아래 명령으로 `.env` 파일 생성 — **PowerShell** 기준

```
# infrastructure/docker/ 로 이동
cd infrastructure/docker

# .env 파일 생성 (Out-File: UTF-8 without BOM)
@"
POSTGRES_USER=kyobo
POSTGRES_PASSWORD=kyobo1234
POSTGRES_DB=kyobo_dev
REDIS_PASSWORD=redis1234
"@ | Out-File -FilePath .env -Encoding utf8NoBOM
```

주의: `.env` 파일이 없으면 `POSTGRES_PASSWORD`가 빈 문자열이 되어 컨테이너 실행이 실패합니다.
생성 후 `cat .env` 로 내용을 반드시 확인하세요.

실습 Step 2 — 컨테이너 실행 + 상태 확인

PostgreSQL + Redis 컨테이너를 백그라운드로 실행합니다

```
# PostgreSQL + Redis 컨테이너 백그라운드 실행
docker compose \
  -f docker-compose.dev.yml \
  up -d postgres redis

# 컨테이너 상태 확인
docker compose -f docker-compose.dev.yml ps
```

- ✓ **postgres** 행 — STATUS 열에 Up (healthy) 출력되면 정상
- ✓ **redis** 행 — STATUS 열에 Up 출력되면 정상

처음 실행 시: 이미지 pull 때문에 1~2분 소요될 수 있습니다. ps 에서 Starting 상태면 잠시 기다린 후 다시 확인하세요.

실습 Step 3 — psql 접속 + 테이블 확인

initdb.d 스크립트가 자동 실행되어 5개 테이블이 생성됩니다

```
# psql 접속 (포트 5433 - 호스트 포트 포워딩)
psql -h localhost -p 5433 \
-U kyobo -d kyobo_dev

# 테이블 목록 확인
\dt
```

정상 출력 — 5개 테이블이 보여야 합니다:

mint_requests · processed_events · outbox_events · user_nft_holdings · audit_log

테이블이 없을 때: 볼륨이 이미 존재하면 initdb.d 가 재실행되지 않습니다.

docker compose down -v 후 up -d 를 다시 실행하세요.

실습 Step 3 — 접속 정보 + 컨테이너 종료

개발 환경 포트 번호와 종료 명령 정리

접속 정보

서비스	호스트	포트	비고
PostgreSQL	localhost	5433	user: kyobo / db: kyobo_dev
Redis	localhost	6380	password: redis1234
Hardhat Node	localhost	8545	별도 실행 시 (M3 이후)

컨테이너 종료

```
# 컨테이너만 종료 - 볼륨(데이터) 유지
docker compose -f docker-compose.dev.yml down

# 볼륨까지 삭제 - initdb.d 재실행 필요할 때
docker compose -f docker-compose.dev.yml down -v
```


실습 트러블슈팅

자주 발생하는 오류와 해결 방법

오류 메시지	원인	해결
docker: command not found	Rancher Desktop이 아직 시작 중이거나 PATH 미등록	System tray에서 Running 확인 후 터미널 재시작
error during connect: pipe ... rancher-desktop	Container Engine이 containerd로 설정됨	Preferences → Container Engine → dockerd (moby) 변경 후 Apply
WSL 배포판 깨짐 / 업데이트 실패	WSL 상태 불안정	wsl --unregister Ubuntu → wsl --install -d Ubuntu 재설치
POSTGRES_PASSWORD must not be empty	.env 파일 없음 또는 값 누락	infrastructure/docker/.env 파일 생성 확인, cat .env로 내용 점검
테이블이 생성되지 않음 (#dt 빈 결과)	볼륨이 남아있어 initdb.d 스크립트가 재실행되지 않음	docker compose down -v 후 up -d 재실행
Input/output error: meta.db	Rancher Desktop 상태 파일 손상	Rancher Desktop Quit → 재실행; 지속 시 앱 재설치

실습 완료 기준

아래 항목을 모두 확인하면 M4 개발 환경 준비 완료

실행 확인 체크리스트

- ☒ `wsl --list --verbose` → Ubuntu STATE=**Running**
- ☒ `docker info` → Server Version 출력
- ☒ `docker compose ps` → postgres, redis **Up (healthy)**
- ☒ `psql \dt` → 5개 테이블 목록 출력
- ☒ `down -v` → `up -d` 재실행 후 테이블 재생성 확인

개념 이해 체크리스트

- ☒ WSL2가 왜 필요한지 설명할 수 있다
- ☒ `dockerd(moby)` vs `containerd` 차이를 안다
- ☒ `down` vs `down -v` 차이를 안다
- ☒ `initdb.d` 스크립트가 언제 실행되는지 안다
- ☒ `.env` 파일 역할 (비밀번호 주입)을 설명할 수 있다